

Análisis Teórico y Experimental de Algoritmos de Divide y Vencerás: Ordenamiento, Búsqueda y Selección

Andres Barbosa[†], Emmanuel Velásquez[†] y Diego Oyarzo[†]

[†]Universidad de Magallanes

Este informe fue compilado el 15 de mayo de 2026

Resumen

El presente proyecto analiza empírica y teóricamente el comportamiento de algoritmos basados en el paradigma de Divide y Vencerás, aplicándolos sobre un robusto sistema de gestión de deportistas construido nativamente en C. El estudio abarca algoritmos de ordenamiento como Merge Sort (en su vertiente clásica y optimizada mediante Insertion Sort) y Quick Sort utilizando el esquema particional de Lomuto con 4 heurísticas de pivote. Asimismo, se incorporan esquemas de Búsqueda Binaria (recursiva y delimitación de rangos), Búsqueda Exponencial y Búsqueda por Interpolación; cerrando el abanico con Quick Select para la búsqueda del k-ésimo elemento.

Mediante evaluaciones sobre variados tamaños de entrada y arreglos dispares, los resultados arrojan evidencia rigurosa sobre el salto cualitativo en la complejidad temporal asintótica (de $\mathcal{O}(n^2)$ a $\mathcal{O}(n \log n)$) en comparación a técnicas sistemáticas previas y explora en profundidad cómo la selección paramétrica del pivote o los umbrales de combinación moldean determinantemente el rendimiento final en escenarios prácticos.

Keywords: Divide y Vencerás, Merge Sort, Quick Sort, Búsqueda Exponencial, Quick Select, Complejidad Temporal, Lenguaje C, Lomuto

■ Índice

1	Introducción	3
2	Objetivo principal	3
2.1	Objetivos secundarios	3
3	Desarrollo	4
3.1	Análisis Teórico de Algoritmos de Ordenamiento	4
	Merge Sort: Clásico y Optimizado • Quick Sort y sus Variantes de Pivote	
3.2	Análisis Teórico de Algoritmos de Búsqueda	5
	Búsqueda Binaria: Recursiva y por Rango • Búsqueda Exponencial • Búsqueda por Interpolación	
3.3	Análisis Teórico de Algoritmos de Selección	6
	Quick Select	
3.4	Resultados Experimentales	7
	Ordenamiento: Mejor, Peor y Promedio • Impacto del Umbral en Merge Sort • Desempeño en Algoritmos de Búsqueda • Selección con Quick Select	

3.5	Aplicación Práctica y Funcionalidades	15
4	Conclusiones	17
5	Contribución por integrante	18
5.1	Andrés Barbosa	18
5.2	Emmanuel Velásquez	18
5.3	Diego Oyarzo	18
6	anexos	19

1. Introducción

El diseño de algoritmos tiene como objetivo encontrar soluciones a problemas de manera eficiente teniendo en cuenta las limitaciones de tiempo y memoria que existen dentro de los sistemas informáticos.

Dos de los problemas más comunes en el día a día de la computación son el **ordenamiento** de datos, la **búsqueda** de información y la **selección** de elementos dentro de estructuras de datos; ambos problemas se pueden afrontar desde distintas miradas, en este caso, centrándonos en el poderoso paradigma de **Divide y Vencerás**.

El presente informe aborda estos problemas sobre una base de datos ficticia de deportistas (generada de manera procedimental) con el fin de exponer, analizar y comparar algoritmos de ordenamiento (*Merge sort* clásico y optimizado, *Quick sort* con múltiples estrategias de pivote), algoritmos de búsqueda (*Búsqueda binaria recursiva*, *exponencial* e *interpolación*) y selección (*Quick select*). Todo ello analizando su comportamiento a nivel teórico y contrastándolo mediante experimentación empírica.

2. Objetivo principal

Analizar y comparar el comportamiento de algoritmos de ordenamiento, búsqueda y selección basados en la estrategia de *Divide y Vencerás*, estableciendo una correspondencia entre los resultados teóricos y experimentales aplicados a un sistema de gestión.

2.1. Objetivos secundarios

1. Comprender el funcionamiento e implementar correctamente algoritmos con estrategia recursiva de partición.
2. Analizar empíricamente la complejidad evaluando el impacto de optimizaciones (ej: umbrales en Merge Sort) y variantes (ej: tipos de pivotes en Quick Sort).
3. Integrar los algoritmos al sistema desarrollado empleando criterios de justificación basados en la eficiencia.
4. Documentar y contrastar de manera gráfica la mejora sustancial en tiempos de ejecución frente a los algoritmos con complejidad asintótica de orden cuadrático de iteraciones previas.

3. Desarrollo

3.1. Análisis Teórico de Algoritmos de Ordenamiento

En esta sección se detallan los algoritmos de ordenamiento implementados bajo el paradigma de *Divide y Vencerás*. Se analizará su funcionamiento de manera teórica, desglosando sus pseudocódigos y estudiando su complejidad asintótica global.

3.1.1. Merge Sort: Clásico y Optimizado

El algoritmo Merge Sort basa su funcionamiento en dividir el arreglo por la mitad repetidamente hasta obtener sub-arreglos de un solo elemento, para luego fusionarlos (operación *merge*) de manera ordenada.

Algorithm 1 Merge Sort Clásico ▷ $V[beg..end]$

```

1: procedure MERGESORT( $V, beg, end$ )
2:   if  $beg < end$  then
3:      $m \leftarrow beg + \lfloor \frac{end-beg}{2} \rfloor$ 
4:     MERGESORT( $V, beg, m$ )
5:     MERGESORT( $V, m + 1, end$ )
6:     MERGE( $V, beg, m, end$ )

```

Análisis de Complejidad: Merge Sort siempre divide el arreglo en dos mitades, tomando tiempo constante $\mathcal{O}(1)$. La función MERGE combina dos sub-arreglos transicionando todos sus n elementos, lo cual toma $\mathcal{O}(n)$. La relación de recurrencia aplicable es:

$$T(n) = 2T(n/2) + \mathcal{O}(n)$$

Por el Teorema Maestro, esto siempre resulta en una complejidad temporal constante de $\mathcal{O}(n \log n)$ en el mejor, peor y caso promedio.

Variante Optimizada con Umbral (Threshold): Al delegar excesivamente al sistema la creación de árboles recursivos inmensos de llamadas y arreglos subyacentes mínimos, se penaliza el rendimiento. Algoritmos de orden cuadrático como *Insertion Sort* poseen gastos constantes diminutos para entradas limitadas. La variante optimizada introduce un límite de recursión (*UMBRAL* o *threshold*); si el tamaño de un sub-arreglo alcanza un $n \leq \text{Threshold}$, se corta la fragmentación y se despacha a la rutina de Insertion Sort, mejorando notablemente los tiempos empíricos a pesar de mantener su categoría perimetral $\mathcal{O}(n \log n)$.

3.1.2. Quick Sort y sus Variantes de Pivote

Quick Sort selecciona un elemento referencial o *pivote* y particiona el arreglo utilizando el esquema de partición original de **Lomuto**. Todos los datos menores que el pivote recaen iterativamente a su lado izquierdo, mientras que los remanentes se asientan a su derecha. Finalmente, propaga sus llamadas lógicas separadamente.

Algorithm 2 Quick Sort Recursivo ▷ Basado en partición de Lomuto

```

1: procedure QUICKSORT( $V, low, high$ )
2:   if  $low < high$  then
3:      $p \leftarrow \text{LOMUTOPARTITION}(V, low, high)$ 
4:     QUICKSORT( $V, low, p - 1$ )
5:     QUICKSORT( $V, p + 1, high$ )

```

Dependiendo de cómo se selecciona el pivote antes de particionar, el algoritmo lidia diferentemente con las anomalías de los datos (como conjuntos ya previamente ordenados). Se implementaron 4 directrices:

- **Último o Primer elemento:** Determinista e inherentemente veloz, pero susceptible y frágil frente a datos ya estructurados.
- **Elemento Aleatorio:** Invoca a los temporizadores nativos en cada paso para elegir el límite. Es probabilísticamente resiliente al *Peor Caso*.
- **Mediana de tres:** Escoge matemáticamente la mediana entre el primer, el central y el último elemento; aislando de forma consistente las desviaciones extremistas.

Análisis de Complejidad Global:

- **Mejor y Caso Promedio** $\mathcal{O}(n \log n)$: Estructura el árbol recursivo de forma simétrica si el pivote divide permanentemente el arreglo en dos particiones proporcionales.
- **Peor Caso** $\mathcal{O}(n^2)$: Ocurre si el esquema particional selecciona ininterrumpidamente un máximo o mínimo. (ej., Pivote del último elemento en arreglos totalmente ordenados).

3.2. Análisis Teórico de Algoritmos de Búsqueda

3.2.1. Búsqueda Binaria: Recursiva y por Rango

Algorithm 3 Búsqueda Binaria Recursiva ▷ $V[beg..end]$ ordenado

```

1: procedure BINSEARCHREC( $V, beg, end, x$ )
2:   if  $beg > end$  then
3:     return  $-1$ 
4:    $m \leftarrow beg + \lfloor \frac{end - beg}{2} \rfloor$ 
5:   if  $V[m] == x$  then
6:     return  $m$ 
7:   else if  $V[m] < x$  then
8:     return BINSEARCHREC( $V, m + 1, end, x$ )
9:   else
10:    return BINSEARCHREC( $V, beg, m - 1, x$ )

```

Al igual que su versión iterativa de la tarea pasada, fragmenta la zona objetivo fraccionalmente. Conlleva una complejidad temporal de $\mathcal{O}(\log n)$ para el peor y caso promedio.

Por otro lado, la variante de **Búsqueda por Rango** aplica este mismo principio logarítmico para dictar umbrales a los topes izquierdo y derecho del arreglo de matches para un solo elemento x , retornando límites masivos usando sólo una superposición asintótica de $\mathcal{O}(\log n)$.

3.2.2. Búsqueda Exponencial

La búsqueda exponencial o *crecimiento galopante* halla una delimitación asumiendo aumentos en potencias de 2 (saltando multiplicativamente índices como 1, 2, 4, 8...), para finalmente ceder la limitación precomputada a una búsqueda binaria tradicional.

Algorithm 4 Búsqueda Exponencial

```

1: procedure EXPSEARCH( $V, n, x$ )
2:   if  $V[0] == x$  then
3:     return 0
4:    $i \leftarrow 1$ 
5:   while  $i < n$  and  $V[i] \leq x$  do
6:      $i \leftarrow i * 2$ 
7:   return BINSEARCH( $V, i/2, \min(i, n - 1), x$ )

```

Posee implicaciones asombrosas a nivel temporal $\mathcal{O}(\log i)$ donde i es el índice del elemento. Conviértendola en un faro excepcionalmente raudo en grandes bases de datos.

3.2.3. Búsqueda por Interpolación

Apoya su funcionalidad interpretando numéricamente el "valor" lógico a buscar intersecándolo en una recta analítica generada entre las fronteras.

Algorithm 5 Búsqueda por Interpolación

```

1: procedure INTERPOLATIONSEARCH( $V, n, x$ )
2:    $low \leftarrow 0, high \leftarrow n - 1$ 
3:   while  $low \leq high$  and  $x \geq V[low]$  and  $x \leq V[high]$  do
4:      $pos \leftarrow low + \lfloor \frac{(x - V[low]) * (high - low)}{(V[high] - V[low])} \rfloor$ 
5:     if  $V[pos] == x$  then
6:       return  $pos$ 
7:     if  $V[pos] < x$  then  $low \leftarrow pos + 1$ 
8:     if  $V[pos] > x$  then  $high \leftarrow pos - 1$ 
9:   return  $-1$ 

```

En el mejor de los casos (distribuciones de datos absolutamente lineales) su comportamiento salta a proezas de orden minúsculo $\mathcal{O}(\log(\log n))$. Su fragilidad (Peor Caso, donde recae en $\mathcal{O}(n)$) yace en su estricta dependencia de si el conjunto del usuario de sistema expone vacíos gigantes de distribución de puntajes.

3.3. Análisis Teórico de Algoritmos de Selección**3.3.1. Quick Select**

Comparte el mismo árbol generativo de Lomuto en Quick Sort. No obstante, al momento de ser partido el hilo, deduce si el objetivo o índice k -ésimo descansó hacia la izquierda o hacia la derecha. Descartando abruptamente la vía que no posee congruencia asintótica con la ubicación del elemento deseado.

Algorithm 6 Quick Select ▷ Búsqueda indexada selectiva

```

1: procedure QUICKSELECT( $V, low, high, k$ )
2:    $p \leftarrow \text{LOMUTOPARTITION}(V, low, high)$ 
3:   if  $p == k$  then
4:     return  $V[p]$ 
5:   else if  $p > k$  then
6:     return QUICKSELECT( $V, low, p - 1, k$ )
7:   else
8:     return QUICKSELECT( $V, p + 1, high, k$ )

```

Dado que se purga un lado de los datos de forma continua y nunca se ordena el 100 % del conjunto:

- **Mejor y Caso Promedio** $\mathcal{O}(n)$: El sub-arreglo se trunca en proporción constante en la media; procesando $n + n/2 + n/4 \dots$, siendo un orden de magnitud lineal.
- **Peor Caso** $\mathcal{O}(n^2)$: Condicionado igualitariamente por los infortunios teóricos perimetrales de un pivoteo desbalanceado.

3.4. Resultados Experimentales

Esta sección contrasta los hallazgos teóricos con simulaciones obtenidas en los datasets estructurados. El proceso se ejecuto con el *smoke* del binario en modo no interactivo, generando tres conjuntos de datos (ordenado, invertido y aleatorio) de $N = 25000$, ejecutando todas las ejecuciones en cadena y luego graficando con Python. Para rigor de una operación sin fallos de computo se decidió este sistema, realizandose secuencialmente las ejecuciones en un único hilo del procesador. La ejecucion completa tomo aproximadamente entre 40 minutos y 1 hora.

El estilo de ejecucion usado en `smoke.c` fue estrictamente secuencial, encadenando cada experimento y su escritura a CSV antes de iniciar el siguiente. Esta causalidad reduce solapamientos temporales y evita el agotamiento de recursos por concurrencia, garantizando estabilidad y trazabilidad en los resultados.

El entorno de ejecucion se resume a continuacion; la configuracion de hardware y sistema operativo se detalla en el registro tecnico adjunto:

1. Sistema operativo: `Linux Mint 22.2 (zara)`
2. Kernel: `6.17.0-23-generic`
3. Arquitectura: `x86_64`
4. CPU: `AMD Ryzen 5 7235HS (4 nucleos / 8 hilos, 1 socket)`
5. Cache L3: `8 MiB`

Las graficas se presentan en escala lineal; en varios casos las curvas se superponen porque los tiempos reales estan en el orden de microsegundos y el redondeo a 10^{-6} s hace que series distintas colapsen visualmente. Esta condicion se verifica en los CSV almacenados en el repositorio.

3.4.1. Ordenamiento: Mejor, Peor y Promedio

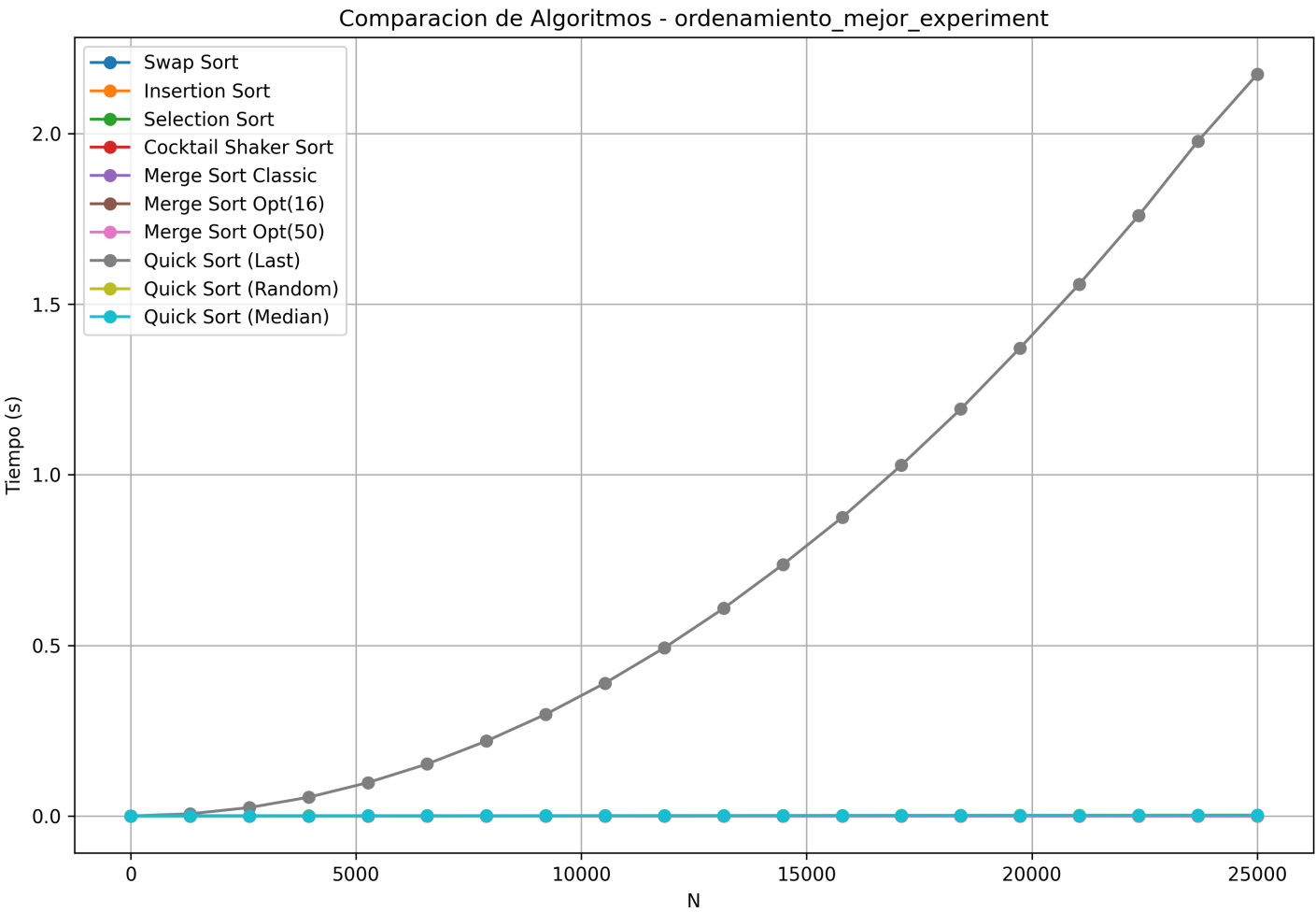


Figura 1. Ordenamiento: mejor caso (arreglo ya ordenado).

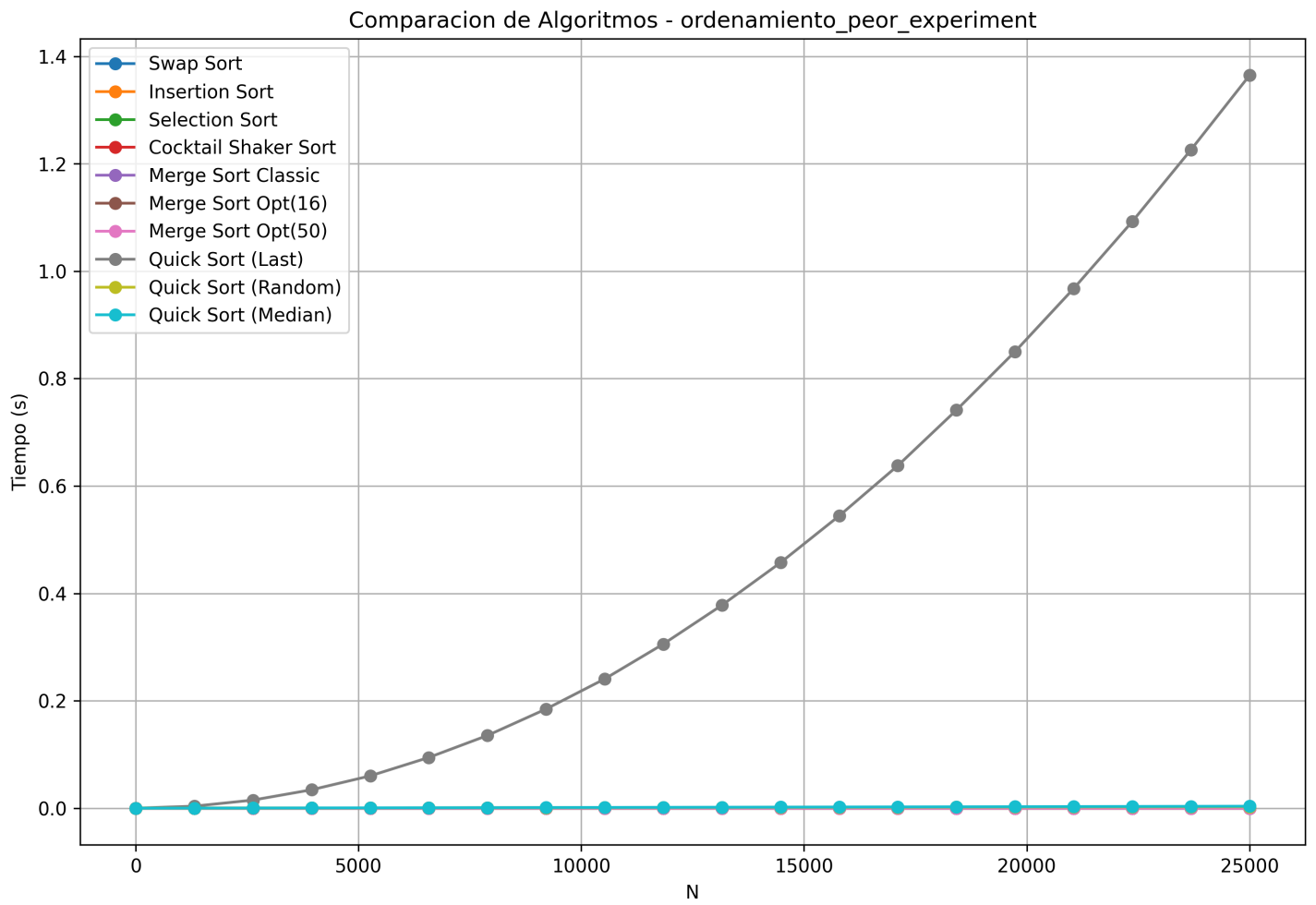


Figura 2. Ordenamiento: peor caso (arreglo invertido).

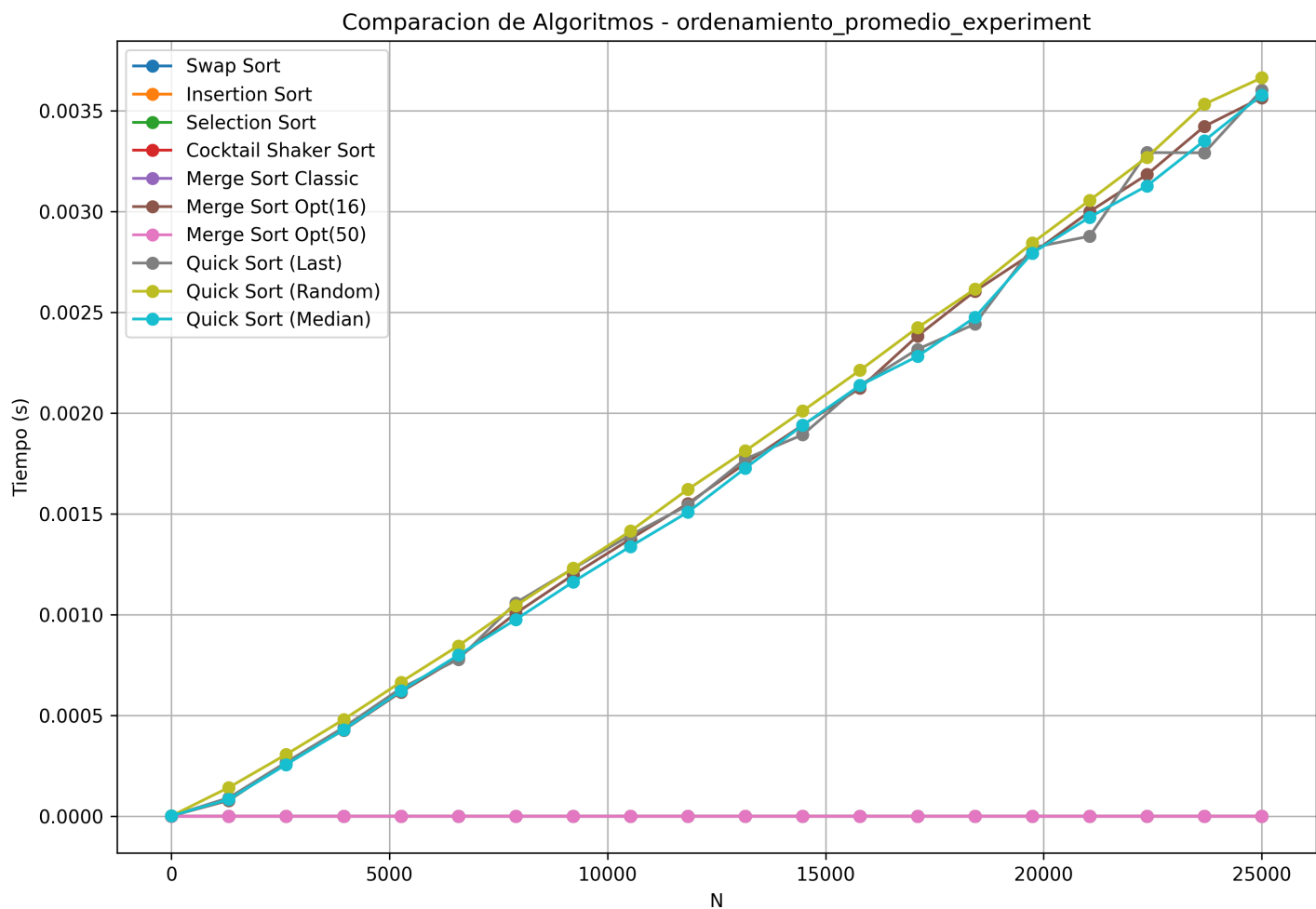


Figura 3. Ordenamiento: caso promedio (arreglo aleatorio).

En los tres escenarios, Quick Sort con pivote aleatorio o mediana se mantiene en el rango bajo de tiempos, mientras que el pivote del ultimo elemento degrada notoriamente en mejor y peor caso (series ya ordenadas o invertidas), exhibiendo el comportamiento esperado de $O(n^2)$. Merge Sort optimizado con umbral 16 sostiene tiempos estables y bajos; el Merge Sort clasico y los algoritmos cuadraticos se acercan a 0.000000 s por efecto del redondeo y la escala, pero sus tendencias se distinguen en el CSV.

3.4.2. Impacto del Umbral en Merge Sort

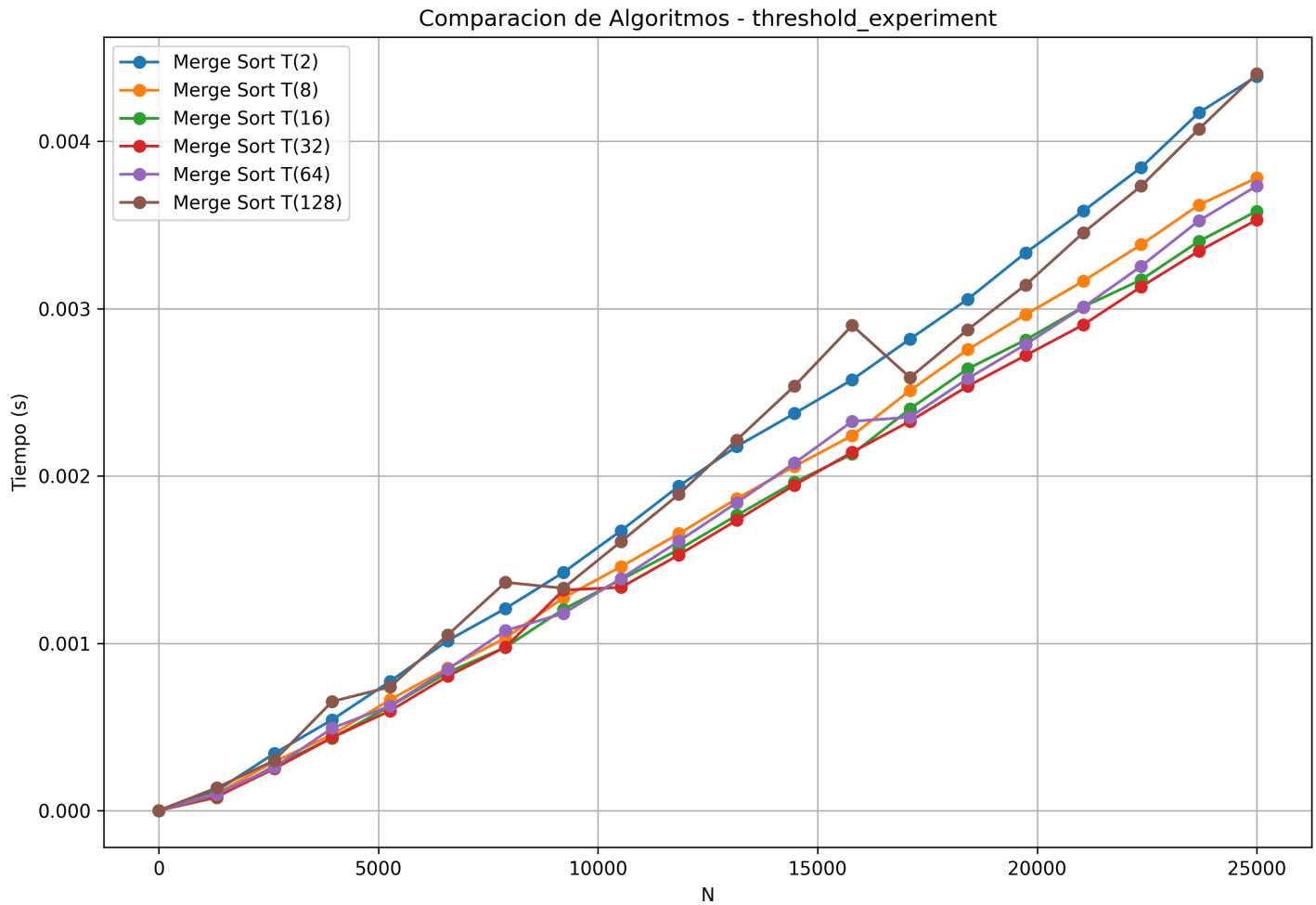


Figura 4. Comparativa de umbrales para Merge Sort optimizado.

La curva muestra que los umbrales intermedios (T(16) y T(32)) presentan el mejor equilibrio en todo el rango de N , evitando el costo de recursion excesiva (T(2)) y el sobrecosto de ordenar subarreglos grandes con Insertion Sort (T(128)). Esto justifica el uso del threshold 16 en las demas pruebas de ordenamiento.

3.4.3. Desempeño en Algoritmos de Búsqueda

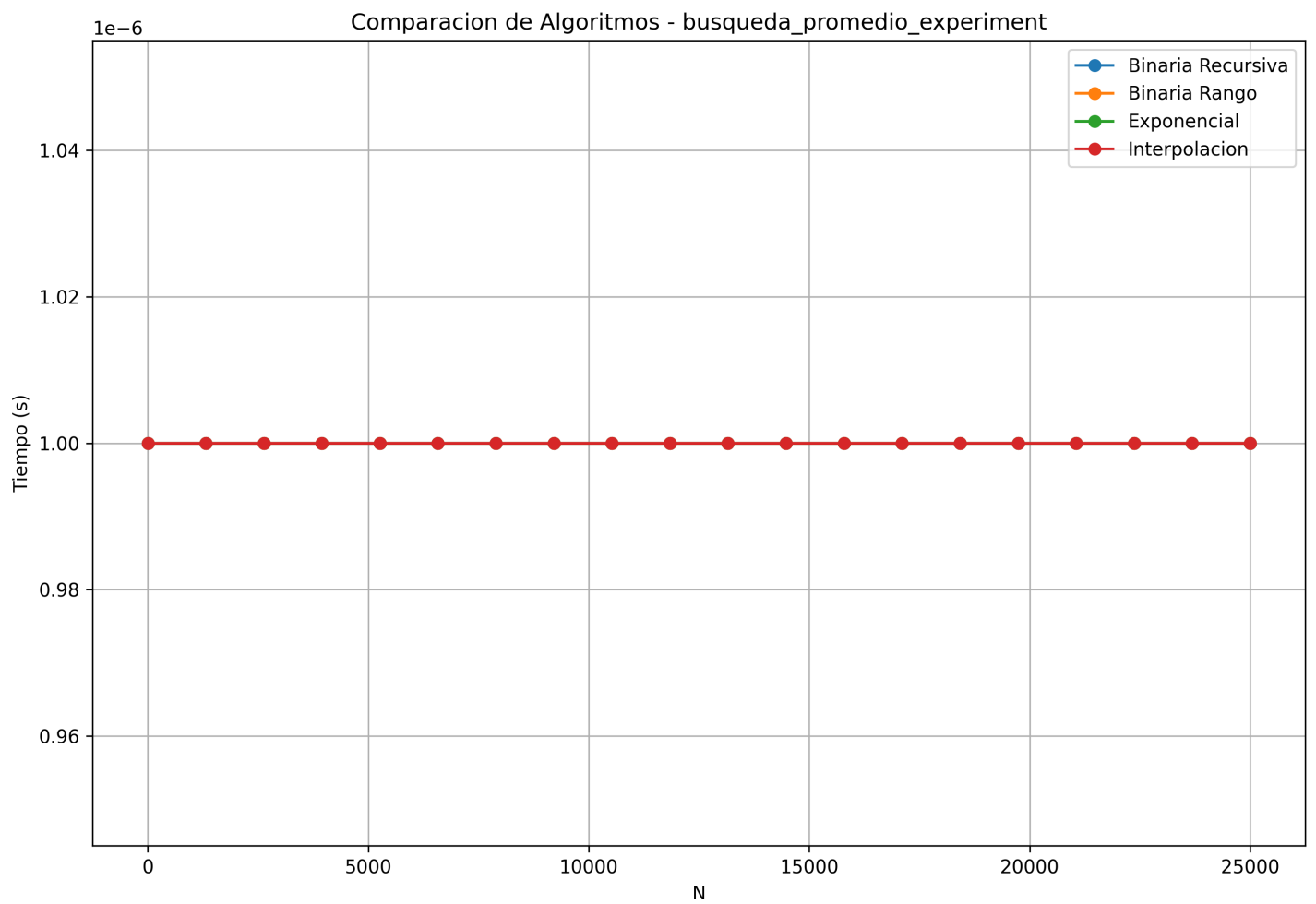


Figura 5. Búsqueda: caso promedio.

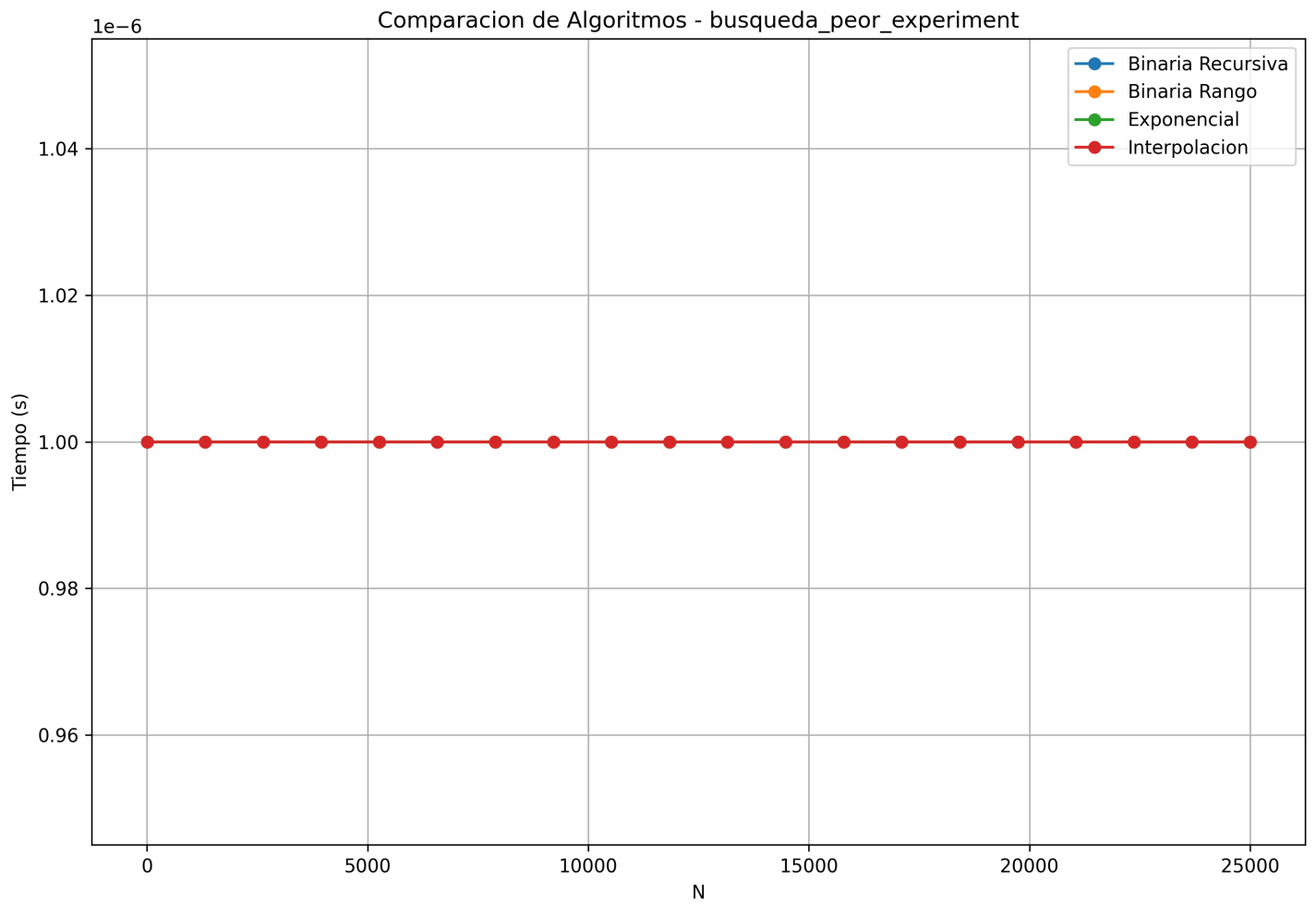


Figura 6. Búsqueda: peor caso.

En ambos casos los tiempos quedan en el orden de 10^{-6} s y se solapan en la grafica. El CSV confirma que todas las variantes se mantienen en valores similares para $N \leq 25000$, lo cual es consistente con complejidades $\mathcal{O}(\log n)$ y un costo constante dominante a este tamaño.

3.4.4. Selección con Quick Select

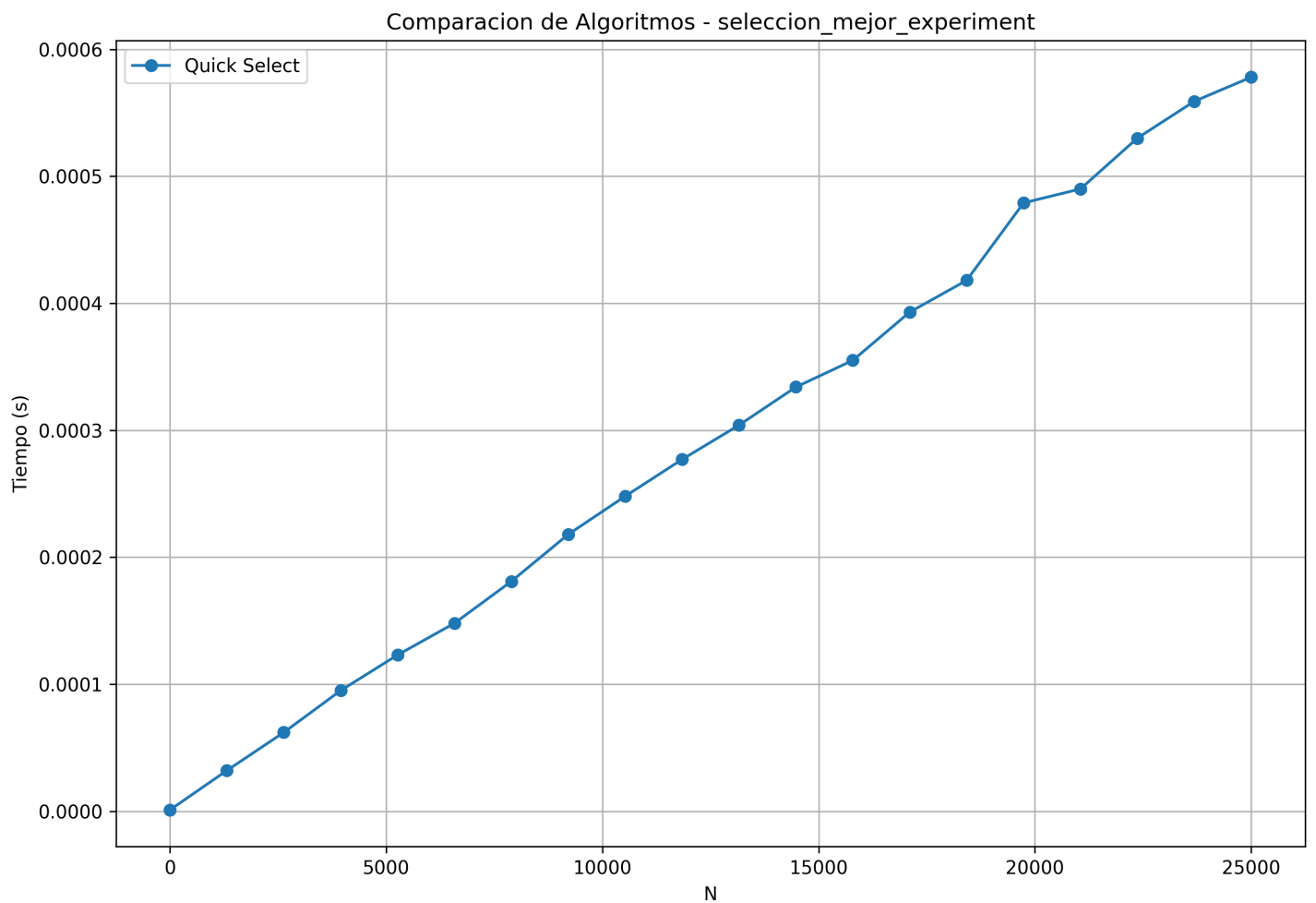


Figura 7. Quick Select: mejor caso (pivote aleatorio en arreglo aleatorio).

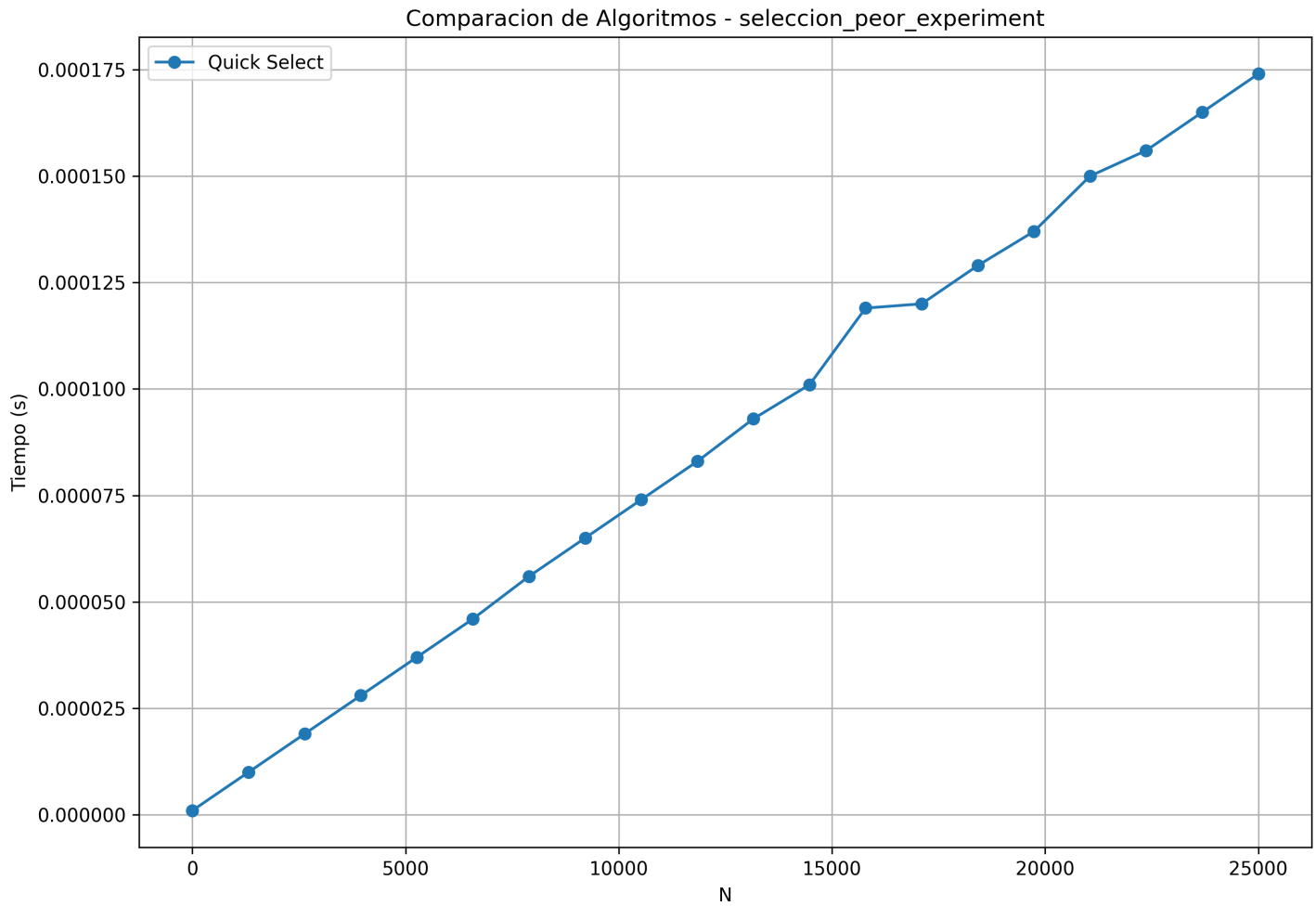


Figura 8. Quick Select: peor caso (pivote ultimo en arreglo ordenado).

Se observa crecimiento aproximadamente lineal en ambos casos, con un peor caso que aumenta mas rapido pero mantiene el rango microsegundo para $N \leq 25000$, coherente con el costo promedio lineal y el peor caso cuadratico amortiguado por el tamaño de entrada y el costo constante de la plataforma.

3.5. Aplicación Práctica y Funcionalidades

El desarrollo técnico del binario interactivo implementó un menú especializado denominado *Data Analytics & Rankings*, mediante el cual los usuarios pueden auditar interactivamente la integridad de los datos de la estructura central de forma abstracta y paginada.

- **Ranking Top N (por Score):** La consola permite generar un escalafón con los mejores atletas. Desde un punto de vista asintótico y práctico, se optó internamente por un **Merge Sort** completo o bien un pre-ordenamiento sistemático, justificándose debido a que el usuario demanda recuperar una fracción subsecuente continua ordenada descendientemente.
- **Listado del K-ésimo Mejor Atleta:** Para exhibir a un único jugador dentro del Top K, se hace uso estricto del algoritmo **Quick Select**. Esto resulta en un ahorro rotundo de tiempo, ya que es innecesario ordenar la totalidad de los datos solo para encontrar el elemento de la posición deseada, promediando un tiempo $\mathcal{O}(n)$.
- **Búsqueda de deportistas por rango de puntos:** La búsqueda de cotas (tanto base como techo) de puntajes se materializa eficientemente a

través de metodologías logarítmicas como **Búsqueda Binaria por Rangos** o **Búsqueda por Interpolación**. En casos donde la distribución del puntaje está altamente uniformada, ambos algoritmos limitan las posiciones de retorno a velocidades de iteración excepcionalmente bajas respecto a un barrido secuencial simple.

Mediante el uso de estas herramientas el usuario visualiza los deportistas paginadamente de a bloques, recibiendo *prints* claros desde el *stdout* con la complejidad aplicada garantizada en segundo plano.

4. Conclusiones

A partir del análisis teórico y experimental realizado sobre los algoritmos de la familia *Divide y Vencerás*, y su contraste con los algoritmos iterativos clásicos, se extraen de manera general las siguientes conclusiones:

El salto de cuadrático a log-lineal

[Aquí se completará tras analizar los gráficos: Se discutirá cómo los algoritmos de ordenamiento $\mathcal{O}(n \log n)$ como Merge Sort y Quick Sort logran reducir masivamente los tiempos de ejecución frente a los de orden $\mathcal{O}(n^2)$ del proyecto anterior, justificando su coste recursivo.]

Optimización empírica: El impacto heurístico

[Aquí se completará tras analizar los gráficos: Se detallará cómo el uso de un umbral en Merge Sort u optimizar la elección del pivote en Quick Sort pueden marcar la diferencia entre un algoritmo promedio y uno altamente eficiente, demostrando que la teoría asintótica se ve directamente afectada por la heurística utilizada y la forma de los datos].

Búsquedas avanzadas

[Aquí se completará tras analizar los gráficos: Se expondrán las ventajas de la búsqueda exponencial para localizar rangos rápidamente en arreglos inmensos y cómo la búsqueda por interpolación rinde dependiendo fuertemente de la distribución de los datos].

Decisiones arquitectónicas del uso de memoria

[Aquí se completará: Breve reflexión sobre el coste espacial. Quick Sort destaca in-place frente a los arreglos auxiliares exigidos por Merge sort, destacando el impacto en entornos con recursos limitados].

5. Contribución por integrante

A continuación se detalla la contribución realizada por cada uno de los integrantes del grupo en el desarrollo del proyecto.

5.1. Andrés Barbosa

Andrés Barbosa — Actualizador y mantenedor principal del proyecto.

- Responsable de actualizar e integrar el código legado proveniente de su grupo anterior.
- Realización de pruebas y validación de las solicitudes del proyecto, asegurando la calidad y funcionalidad del código.
- Liderazgo en asignación de tareas y coordinación del equipo, facilitando la comunicación y colaboración entre los integrantes.

5.2. Emmanuel Velásquez

El estudiante **Emmanuel Velásquez** contribuyó al proyecto con las siguientes tareas:

- Actualización de documentación del repositorio acorde a Proyecto 2 asignado por la docente.
- Prototipado del informe, heredando formato anterior y adaptandolo a los requerimientos del proyecto.
- Revisión de salud de repositorio, constancia de que mantiene archivos correspondientes al día para un correcto uso de git.

5.3. Diego Oyarzo

El estudiante **Diego Oyarzo** contribuyó al proyecto con las siguientes tareas:

- Corrección de errores en análisis de algoritmos.
- Inspector de código, análisis de algoritmos y pruebas.

6. anexos

```
1 /**
2  * @brief Esquema de particion de Lomuto
3  */
4 static int lomuto_partition(Player V[], int low, int high, int pivot_type, int (*comp_f)(Player *, Player *)) {
5     int p_idx = select_pivot_index(V, low, high, pivot_type, comp_f);
6     Player pivot = V[high];
7
8     int i = low - 1;
9     for (int j = low; j < high; j++) {
10         if (comp_f(&V[j], &pivot) <= 0) {
11             i++;
12             swap_player(&V[i], &V[j]);
13         }
14     }
15     swap_player(&V[i + 1], &V[high]);
16     return i + 1;
17 }
```

Código 1. Esquema de partición Lomuto empleado como base en Quick Sort y Quick Select

■ Referencias

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical algorithms*, 3.^a ed. Addison-Wesley, 1997, ISBN: 978-0-201-89684-8. (visitado 13-11-2019).
- [2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2.^a ed. Addison-Wesley, 1998, ISBN: 978-0-201-89685-5. (visitado 02-04-2026).
- [3] Linux Kernel Organization, *Linux Programmer's Manual: QSORT(3)*, Accedido el 12 de abril de 2026, 2024. dirección: <https://man7.org/linux/man-pages/man3/qsort.3.html>.